



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Teste de Software

Relatório de Test Pattern

Kayler de Freitas Moura

Belo Horizonte

2026

1. Padrões de Criação de Dados (Builders)

Por que o CarrinhoBuilder foi usado em vez de um CarrinhoMother?

O padrão Object Mother é ideal para objetos simples ou cujas variações de criação são limitadas, por exemplo, um usuário com tipos definidos como PADRAO e PREMIUM. No entanto, um carrinho de compras (Carrinho) é um objeto complexo que pode conter diferentes listas de itens, interagir com vários tipos de usuário, e possuir diferentes características dinâmicas. Utilizar um CarrinhoMother obrigaria a criação de um método para cada pequena variação, gerando um excesso de métodos e muita rigidez. Já o CarrinhoBuilder resolve isso oferecendo uma interface flexível (chamadas encadeadas de métodos, *fluency*) para montagem dinâmica das instâncias usando valores *default* para os atributos que não nos interessam para testar um determinado cenário.

Exemplo Antes e Depois

Antes (Setup Manual Complexo e Frágil):

JavaScript

```
const usuario = new User(2, 'Usuario Premium',  
  'premium@email.com', 'PREMIUM');  
const item1 = new Item('Celular', 2000);  
const item2 = new Item('Fone de Ouvido', 200);  
const carrinho = new Carrinho(usuario, [item1, item2]);
```

Aqui o teste se suja tendo que lidar com atributos irrelevantes (nomes de usuário, ids, listas manuais com múltiplas dependências importadas num mesmo arquivo). Causa o Test Smell **Setup Obscuro**.

Depois (Setup simples usando Builder):

JavaScript

```
const usuarioPremium = UserMother.umUsuarioPremium();  
  
const carrinho = new CarrinhoBuilder()  
  .comUser(usuarioPremium)  
  .comItens([new Item('Item Caro', 200)])  
  .build();
```

Como o Builder melhora a legibilidade e manutenção do teste

O Data Builder encapsula o processo ruidoso da instanciação, além de abstrair importações desnecessárias do cenário. Se novas regras ou propriedades surgirem no domínio Carrinho, alteramos e refatoramos as regras exclusivamente em CarrinhoBuilder.js, preservando todos os testes de sofrerem o impacto ou de quebrarem à toa, e diminuindo assim os "Testes Frágeis". É mais legível pois a construção do objeto torna as intenções óbvias descrevendo exatamente o que importa na leitura.

2. Padrões de Test Doubles (Mocks vs. Stubs)

Teste de "sucesso Premium" da Etapa 5

Nesse teste testamos um cliente premium que finaliza uma compra. As dependências foram tratadas da seguinte forma:

- **GatewayPagamento**: atua como Stub.
- **PedidoRepository**: atua como Stub, ou dummy.
- **EmailService**: atua como Mock.

GatewayPagamento como Stub vs EmailService como Mock

O GatewayPagamento foi configurado e usado principalmente como um Stub. A nossa principal preocupação era simplesmente *prover uma resposta pré-determinada* (que o pagamento funcionou: { success: true }) para conduzir os fluxos internos de método do CheckoutService no cenário feliz (o "Estado"), portanto aplicamos Verificação de Estado. A execução não deveria quebrar ali para testar a finalidade do comportamento.

O EmailService, em oposição, não altera o fluxo principal do CheckoutService nem seu sucesso se o email for enviado ou não (conforme as regras). Ele representa um efeito colateral cujo único intuito de fato é a chamada e comunicação final daquele cenário com um outro sistema periférico. Para avaliarmos se esse "efeito colateral" aconteceu, precisamos aplicar a Verificação de Comportamento, conferindo se o envio foi efetivamente disparado (expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);) e com quais atributos (toHaveBeenCalledWith(...)), o que classifica o EmailService em nosso teste como um Mock.

Testes passando:

```
test-pattern git:(main) ✗ npm test

> test-pattern@1.0.0 test
> jest

PASS __tests__/CheckoutService.test.js
  CheckoutService
    quando o pagamento falha
      ✓ deve retornar null (4 ms)
    quando um cliente Premium finaliza a compra
      ✓ deve aplicar desconto e enviar email (2 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        0.679 s, estimated 1 s
Ran all test suites.
test-pattern git:(main) ✗
```

3. Conclusão

O uso equilibrado dos Padrões de Teste tanto para criação de estado e de dados (Data Builders e Object Mothers) como de controle de dependências (Test Doubles como Mocks e Stubs), ataca simultaneamente diversos perigos críticos de acoplamento e legibilidade do código.

Evitamos instabilidades decorrentes de requisições de rede, efeitos colaterais lentos (Stubs e Mocks) e conseguimos nos livrar de Test Smells como fragilidade nos testes, código sujo ou setups enormes, resultando de fato numa suíte limpa, fluente, autossuficiente e altamente sustentável a longo prazo.